

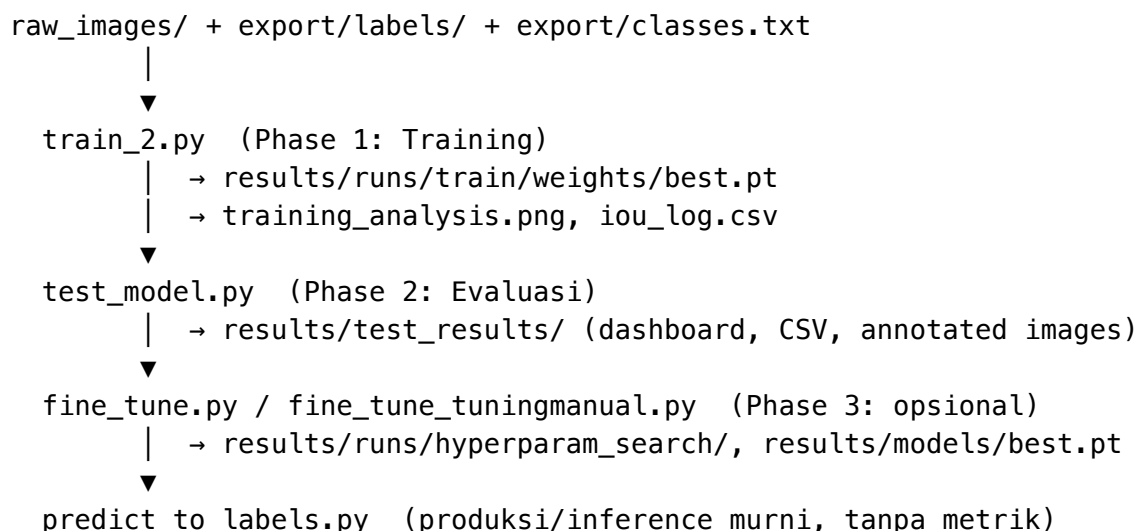
Detail Teknis — YOLO Bukukia Training Pipeline

1. Alur Sistem
2. `config.py` — Konfigurasi Terpusat
3. Script Inti
 - 3.1 `train_2.py` — Training
 - 3.2 `iou_callback.py` — Tracking Mean IoU per Epoch
 - 3.3 `test_model.py` — Evaluasi (Phase 2)
 - 3.4 `predict_to_labels.py` — Inference Produksi
 - 3.5 Fine-Tuning: dua pendekatan berbeda
4. `analyze.py` — Analisis Pasca-Training
5. `run_pipeline.py` — Orchestrator Parsial
6. Script Evaluasi IoU — Perbandingan
7. Format Label & Kelas
8. Dua Folder Dataset
9. Utilitas Manajemen Data (Colab)
10. Utilitas Lain
 - `extract_label_details.py`
 - `generate_report.py`
11. Referensi Path Penting
12. Catatan untuk Pengembangan Selanjutnya

Detail Teknis — YOLO Bukukia Training Pipeline

Untuk AI Engineer / Data Engineer yang butuh detail implementasi baris kode.
Ringkasan per role (PM, UI/UX, Security, DevOps, dst) ada di [../00-concept/README.md](#).

1. Alur Sistem



run_pipeline.py bukan orchestrator penuh — lihat §5.

2. config.py — Konfigurasi Terpusat

- Deteksi environment otomatis (Colab / macOS-MPS / Linux-CUDA / lainnya-CPU) → menentukan DEVICE dan WORKERS.
- Semua hyperparameter bisa di-override lewat environment variable.

Default training: | Var | Default | |---| | YOLO_MODEL | yolo26n.pt | | YOLO_EPOCHS | 80 | | YOLO_IMGSZ | 640 | | YOLO_CONF | 0.25 | | YOLO_OPTIMIZER | AdamW | | YOLO_LR0 / YOLO_LRF | 0.008 / 0.01 | | YOLO_COS_LR | True | | YOLO_PATIENCE | 25 | | YOLO_RESUME | False |

Default fine-tune: YOLO_FT_EPOCHS=20, YOLO_FT_LR0=0.001, YOLO_FT_FREEZE=10.

Catatan: YOLO_FT_FREEZE didefinisikan tapi tidak dipakai di fine_tune.py manapun — kemungkinan sisa config yang sudah tidak aktif.

Path helper penting:

- get_latest_train_run() (config.py:101–110) — otomatis memilih folder train* termuda di results/runs/ sebagai TRAIN_RUN aktif. Script evaluasi lain memakai ini secara default kalau tidak diberi --model eksplisit.
- get_best_model() (config.py:127–135) — fallback cascade: TRAIN_RUN/weights/best.pt → results/models/best.pt → input_files/training_models/best.pt.

3. Script Inti

3.1 train_2.py — Training

Argumen: --model, --epochs, --imgsz, --batch, --limit/--fraction, --images-dir, --split-dir, --fl-gamma (default 1.5), --fl-alpha (default 0.25).

Alur:

1. reset_dataset() — hapus & buat ulang results/dataset/.
2. prepare_dataset() — pairing gambar↔label berdasarkan nama file (stem), strip prefix xxx__ jika ada, split 80/20 train/val, seed=42. Jika --split-dir diberikan, pakai use_presplit_dataset().
3. create_data_yaml() — baca classes.txt, generate data.yaml untuk Ultralytics.
4. run_training() — training YOLO via Ultralytics.

Focal Loss (train_2.py:232–243): diinjeksi via callback on_pretrain_routine_end yang mengganti trainer.criterion.bce menjadi ultralytics.utils.loss.FocalLoss(gamma, alpha). Set --fl-gamma 0.0 untuk kembali ke BCE standar.

IoU callback: diregistrasi via model.add_callback("on_fit_epoch_end", iou_cb.on_epoch_end) — lihat §3.2.

`train_kwargs` yang di-fix di kode (`train_2.py:273–292`): `augment=False`, `warmup_epochs=3`, `close_mosaic=15`.

Output: `results/runs/train/weights/best.pt`, `training_analysis.png`, `iou_log.csv`. Setelah training selesai, otomatis memanggil `analyze.py::run_analysis()` dan `log_training_history()`.

3.2 `iou_callback.py` — Tracking Mean IoU per Epoch

Kelas `IoUTracker`. Setiap `on_epoch_end`:

1. Coba ambil Mean IoU langsung dari `trainer.validator.stats["tp"]`.
2. Fallback (`iou_callback.py:49–78`) jika data itu tidak tersedia: estimasi heuristik dari rasio `mAP50–95 / mAP50` dengan formula $0.5 + \text{ratio} \times 0.4$. Ini bukan IoU asli, hanya perkiraan kasar.

Menulis `iou_log.csv` secara incremental (append per epoch).

3.3 `test_model.py` — Evaluasi (Phase 2)

Argumen: `--conf`, `--model`, `--data`, `--iou-threshold` (default `0.5`).

Untuk tiap gambar: jalankan model → load GT label → hitung IoU manual (`compute_iou`) → matching prediksi ↔ GT secara greedy by highest IoU descending → klasifikasi TP/FP/FN/TN per deteksi & per gambar.

Output ke `results/test_results/` (auto-increment jadi `test_results1`, dst):

- `test_dashboard.png`, `iou_distribution.png`, `test_report.txt`
- CSV: `test_per_detection.csv`, `test_per_image.csv`, `test_per_class_metrics.csv`
- `annotated/` — gambar dengan bbox+label TP/FP
- `predicted_labels/` — label YOLO hasil prediksi (kolom ke-6 = confidence)

3.4 `predict_to_labels.py` — Inference Produksi

Jalankan model pada folder gambar, tulis `.txt` label YOLO ke `--out` (opsional `--save-conf`). Tidak menghitung metrik apa pun — beda dari `test_model.py` yang butuh ground-truth. Ini script yang dipanggil `ocr-kia-reader/infra/nightly-pipeline/` untuk auto-label produksi.

3.5 Fine-Tuning: dua pendekatan berbeda

`fine_tune.py` — pakai `model.tune()` bawaan Ultralytics (algoritma genetik). Output: `results/runs/hyperparam_search/best_hyperparameters.yaml`, `best_params.json`; model final di-copy ke `results/models/best.pt`.

`fine_tune_tuningmanual.py` — coordinate descent manual: sweep `lr0` lalu `lrf` (grid log-scale, default 10 steps, `PARAM_ORDER = ["lr0", "lrf"]`). Tiap kombinasi dilatih singkat (`--trial-epochs`, default 10), ambil mAP50 terbaik, fix, lanjut. Output: heatmap PNG, `trial_results.csv`, `best_params.json`.

Kedua script fine-tune independen — pilih salah satu.

4. analyze.py — Analisis Pasca-Training

Dipanggil dari train_2.py dan run_pipeline.py.

- `load_results()` — baca `results.csv` (native Ultralytics) + `iou_log.csv`.
- `detect_plateau()` — smoothing moving-average (window=5) + deteksi gradient mendekati nol (threshold 0.005) pada mAP50; plateau valid jika durasi ≥ 5 epoch.
- `early_stopping_analysis()` — cari best epoch berdasarkan mAP50 tertinggi.
- `diagnose_bottleneck()` — heuristik rule-based: dataset kecil (<100 gambar), overfitting ($\text{val_loss} > 1.5 \times \text{train_loss}$), underfitting ($\text{train_loss} > 1.0 \rightarrow$ saran upgrade model n $\rightarrow$ s/m), recall rendah (<0.5), bbox tidak presisi (mAP50 tinggi tapi mAP50-95 rendah), atau "Good Performance" (mAP50 ≥ 0.8).
- `plot_training_analysis()` — dashboard 9-panel $\rightarrow$ `training_analysis.png`.
- `log_training_history()` + plot terkait — catat riwayat training lintas run ke `verification/training_history.csv`.

5. run_pipeline.py — Orchestrator Parsial

Hanya menjalankan:

1. `analyze.py` (default TRAIN_RUN)
2. `test_model.py` (default)
3. `fine_tune.py` — opsional, hanya jika flag `--fine-tune` diberikan

`train_2.py` TIDAK dipanggil — training harus manual dulu.

⚠ **Bug potensial** (`run_pipeline.py:222–231`): argumen `--trial-epochs/--steps` diteruskan ke `fine_tune.py` saat `--fine-tune` dipakai, padahal `fine_tune.py` menerima `--tune-epochs/--iterations`. Argumen itu milik `fine_tune_tuningmanual.py` yang tidak dipanggil di sini.

Setelah selesai, generate laporan HTML self-contained (Report Detail_<timestamp>.html, gambar embed base64). Ada seksi "verification & retrain" yang mencari kata kunci di log, tapi tidak ada step yang menghasilkan log itu — placeholder untuk fitur Label Studio yang sudah di-remove (lihat changelog README).

6. Script Evaluasi IoU — Perbandingan

Script	Split	Metrik	
<code>evaluate_iou_train.py</code>	<code>images/train</code> (fallback root)	IoU per-deteksi saja, tanpa threshold gating. Tidak ada precision/recall/F1	CSV per-image <code>iou_distrib</code>
<code>evaluate_iou_full_train.py</code>	<code>images/train</code> eksplisit	TP/FP/FN/TN lengkap, <code>--iou-threshold</code>	3 CSV + 4 plc

Script	Split	Metrik	
		(default 0.5), precision/recall/F1	
evaluate_iou_full.py	images/val (default)	Sama dengan di atas, target VAL	3 CSV + 4 plc

evaluate_iou_full_train.py dan evaluate_iou_full.py adalah script kembar (~95% identik) — beda split target, dipakai berpasangan untuk cek overfitting (train vs val).

7. Format Label & Kelas

Format YOLO: <class_id> <x_center> <y_center> <width> <height>, ternormalisasi 0–1. Kolom ke-6 opsional untuk confidence.

classes.txt aktif, 6 kelas index 0–5:

```
0: bukukia2024_page0_segment1
1: bukukia2024_page0_segment2
2: bukukia2024_page0_segment3
3: bukukia2024_page0_segment4
4: bukukia2024_page0_segment5
5: 2024_nik
```

⚠ **Inkonsistensi:** filter_labels_by_segment_colab.py (scripts/filter_labels_by_segment_colab.py:6–21) memakai daftar CLASSES_2023_2024 dengan ID lebih besar (3, 4, 27–38), termasuk kelas 2023_nik dan bukukia2023_page0_segment1–7 yang tidak ada di classes.txt saat ini. Kemungkinan sisa fase data-cleaning sebelumnya. Jangan pakai tanpa verifikasi ulang mapping.

8. Dua Folder Dataset

- **input_files/** — dataset aktif. raw_images/ 91 foto (pola nama: taskXXXXX_<kodebook>_KIA_<nama_ibu>_<nama_ayah>_<nik>_<lingkungan>_<kelurahan>_<kecamatan>_page1.png), export/labels/, export/classes.txt, test-dataset/images/ (5 foto test).
- **input_files_kia2024/** — raw_images/ kosong, hanya export/labels/ + export/classes.txt + export/gcs_manifest.txt (peta GCS asli, bucket test-kia-legacy-cover).

Kesimpulan: arsip label-only untuk batch KIA 2024 legacy, foto mentahnya tidak ada di repo lokal.

Catatan PII: nama file di raw_images/ sudah membocorkan nama ibu/ayah dan NIK anak, bukan cuma isi fotonya (lihat contoh di [00-concept/README.md §Security](#)). Folder ini dan results/ sekarang dikecualikan lewat root .gitignore (ditambahkan Agustus 2026 — sebelumnya repo tidak punya .gitignore sama sekali).

9. Utilitas Manajemen Data (Colab)

Murni CLI file-management, tidak bergantung pada `config.py`.

Script	Fungsi
<code>copy_images_from_labels_colab.py</code>	Match gambar berdasarkan stem nama file label → copy ke folder tujuan
<code>filter_images_colab.py</code> / <code>filter_labels_colab.py</code>	Baca CSV (kolom filename) → copy gambar/label yang cocok
<code>filter_labels_by_segment_colab.py</code>	Copy label yang hanya berisi class ID 2023/2024 legacy — lihat §7
<code>move_random_images_colab.py</code>	Sampling acak N gambar (+ label), mode move/copy, --dry-run, --seed (default 42)

10. Utilitas Lain

`extract_label_details.py`

Baca semua `.txt` label, gabungkan dengan `classes.txt`, ekstrak nama file, nama class, "bookyear" (regex 4 digit), koordinat YOLO mentah → CSV. Untuk audit distribusi label per tahun/kelas.

`generate_report.py`

Generate `results/END_TO_END_REPORT.md` dari `verification/csv_summary/metrics_summary.csv` (grade Trusted/Review/Retrain) + hasil fine-tune + info model.

⚠ Changelog README (v2.3) menyebut fitur ini "Removed dari notebook flow". Dependency yang dirujuk (`verify_segmentation.py`, `csv_summary/`) tidak ditemukan di `scripts/` — kemungkinan dead code dari fase Label Studio feedback-loop lama.

11. Referensi Path Penting

Topik	Path
Semua path & hyperparameter	<code>scripts/config.py</code>
Injeksi Focal Loss	<code>scripts/train_2.py:232–243</code>
Estimasi Mean IoU (fallback heuristik)	<code>scripts/iou_callback.py:49–78</code>
Daftar class ID legacy tidak konsisten	<code>scripts/filter_labels_by_segment_colab.py:6–21</code>

Topik	Path
Potensi bug argumen CLI fine-tune	scripts/run_pipeline.py:222-231
Bukti sumber data KIA 2024 legacy (GCS)	input_files_kia2024/export/gcs_manifest.txt

12. Catatan untuk Pengembangan Selanjutnya

- Verifikasi ulang `--fl-gamma/--fl-alpha` default terhadap hasil training aktual — Focal Loss aktif default (`gamma=1.5`).
- Kalau mau pakai `--fine-tune` di `run_pipeline.py`, cek dulu argumen yang diteruskan (§5).
- `YOLO_FT_FREEZE` di `config.py` tidak dipakai — hapus atau implementasikan.
- Verifikasi ulang mapping `CLASSES_2023_2024` sebelum pakai `filter_labels_by_segment_colab.py`.
- `generate_report.py` kemungkinan dead code — konfirmasi ke pemilik project sebelum dihapus/diaktifkan kembali.